Fakturama Image-to-Cash Automation — Design Document

Part 1 deliverable. This describes how I would build the system before writing any code. Implementation status, deviations found during the build, and evidence are in the README.

1. What the system has to do

Input: one order image. Output: one saved Order and one linked Invoice in Fakturama, with the Debtor and every Product either selected from existing master data or created, the correct VAT rate attached, and the extracted payment status applied. Everything goes through Fakturama's own desktop UI, with no database or file-level shortcuts, and nothing may depend on fixed screen coordinates or a particular window layout.

Two constraints shape the design more than anything else. First, the flow is Order-first: the New Order editor is opened early and stays open while missing master data is created elsewhere, and the Order's own Debtor and Product selectors are the existence checks. Second, every step is verified before the next one starts, and anything ambiguous stops for manual review instead of guessing.

2. Overall shape

I would split the system into four parts that do not know about each other's internals:

1. **Extraction** — turns the image into a typed order model (header, debtor, payment, items).
2. **Validation** — checks the model against itself (arithmetic, uniqueness, required fields) before the UI is touched.
3. **UI layer** — finds Fakturama controls by meaning, performs an action, and reads the result back.
4. **Flow** — the PDF's step sequence, written as guarded stages that call the UI layer and refuse to continue when a read-back does not match.

Source data and observed UI state are kept separate throughout: the flow writes a value from the model, reads what the application now shows, and compares. A value is only treated as "set" when the read-back agrees.

3. Image-extraction strategy

OCR first, LLM as fallback. The source is a printed business document with a stable structure, so I would start with local OCR (Tesseract) rather than an LLM. It is deterministic, runs offline, keeps the customer data on the machine, and its mistakes are the kind that arithmetic validation catches. An LLM-vision call would be the fallback for layouts the OCR parser does not recognise, feeding the same typed model so the validator and the UI flow do not care which extractor produced the data.

Layout handling. Upscale the image (around 3x) before OCR, keep word-level bounding boxes, and group words into lines by vertical position. Use the document's own headings as anchors ("Order Date", "External Reference", "Bill To", "Ship To", "Payment", the items table header) and read values relative to those anchors rather than at fixed pixel positions. The two address blocks sit side by side, so split them on the column gap and on the postal-code line of each block.

Normalisation. Dates to ISO, money to `Decimal` with two places, percentages to integers, the payment method and paid status to a small set of known labels. Item rows come from the table region: SKU, description, quantity, unit net price, VAT %, discount %, line total.

Validation before any UI action. Each line's net must equal quantity × unit net × (1 − discount/100), the lines must sum to the printed net total, VAT and gross must reconcile, SKUs must be unique, a PAID status must come with a payment date, and the currency must match the workspace. Product master gross price is unit net × (1 + VAT/100), never reduced by the

line discount, as the PDF specifies. Any mismatch is reported with the raw OCR text and the run stops; a wrong value typed into Fakturama is far more expensive than a stopped run.

4. Control-discovery and grounding strategy

UI Automation as the primary channel. Fakturama is an Eclipse RCP / SWT application, and SWT exposes most controls through Microsoft UI Automation: toolbar buttons, editor tabs, labels, text fields, combo boxes, check boxes, and the tree in the left panel. I would attach to the exact Fakturama window by title and process, then find controls by what they mean, not where they are:

- **Editors** by their tab text ("New Order", "New Debtor", "New product", "New Invoice").
- **Fields** by their label: the Edit or ComboBox immediately to the right of a Text element whose name matches ("Date", "Cust.Ref.", "Company", "Item Number"), searched only inside the active editor's bounds so an identically named field in another tab cannot be picked up.
- **Toolbar actions** ("Order", "Save", "Invoice") by name.
- **Dialogs** ("Select the address", "Select a product") by title.

This gives semantic identity with no coordinates. If a label or control type changes between Fakturama versions, the lookup fails loudly rather than clicking the wrong thing.

Input the way a user does it. SWT widgets often ignore UIA's programmatic set-value, so values are entered with real keyboard input (select all, type, Tab) and combo items are chosen by opening the list and selecting the visible item. Every entry is read back through UIA afterwards. The date field is a Nebula CDateTime segmented widget; I would drive it segment by segment from the keyboard and confirm the displayed date before moving on.

Where UIA runs out. Fakturama's grids — the items table in the Order editor, the rows in the two selector dialogs, and the Data > Documents list — are NatTable controls, which render as a single canvas and expose no cells. For these I would locate the table region from UIA-grounded neighbours (the dialog bounds, the search box, the column header), capture that region, and OCR it. Exact-match rules make this safe: a Debtor row counts only when Company, First Name, Name, ZIP and City all match; a Product row only on the exact SKU. If OCR produces more than one candidate row or an unreadable one, the stage stops. An empty OCR result is never treated as "not found" unless the dialog also reports no results.

Screen hygiene. Input is only sent while the Fakturama window owns the foreground, captures are taken of the window's own rendered content so a notification or another window on top does not pollute an OCR read, and a lock prevents two runs driving the same desktop.

5. Flow and per-step verification

The flow follows the PDF's numbered steps, each as a stage that must pass its check:

- **Open the Order (1.3–1.8):** click Order, wait for the editor, set Date and Cust.Ref., set Net / With VAT; read all three back.
- **Debtor (2.1–2.13):** open the address selector from the Order, search by company, apply the exact-match rule. If found, select and confirm both addresses populated. If not, Cancel, open New Contact from the left panel, fill identity, main address with Invoice (and Delivery, if identical) role, Miscellaneous (alias, 0% discount, Net), and the payment method — creating it under Data > terms of payment first if it does not exist, with the code mapping from the PDF. Save once, return to the Order, re-select, and confirm the addresses.
- **Products (3.1–3.17), one item at a time in source order:** open the product selector, search the exact SKU. If missing, check Data > VATs for "VAT nn%" with value nn and code S, create it if absent, then New product with SKU, name/description, computed gross price, cost 0, the exact VAT, stock 0. Save once, re-select from the Order, then set Qty, confirm U.Price and VAT, set the line discount, confirm the line price.

- **Save the Order (4.1–4.5):** confirm addresses, lines and totals against the model, Save once, confirm the row in Data > Documents (number, date, Cust.Ref., open state, total).
- **Invoice (4.6–5.7):** use the saved Order's follow-up "Invoice" action, not the toolbar, so the link is preserved. Confirm the copied fields, set the payment method, apply paid / payment date / value only if the image says PAID, Save once, confirm both rows in Data > Documents.

One design rule covers the "resolve or create" branches: creation is attempted only after the selector returned no exact row, and success is confirmed by selecting the new record *from the Order's selector*, not by trusting the editor's Save.

6. Failure handling and idempotence

Every stage is fail-closed. A failed read-back, an ambiguous selector result, a total that does not reconcile, or a control that cannot be found stops the run with a structured report and a screenshot of the state at that moment. Nothing is retried blindly, because a retry after a half-applied step risks a duplicate record.

Each record (payment method, Debtor, VAT, Product, Order, Invoice) is saved exactly once. Before each Save the run writes a marker to disk; if the run is restarted, it first searches Data > Documents by the external reference and either resumes against the existing Order or stops, rather than creating a second one.

7. Trade-offs

- **UIA + OCR versus pure vision.** UIA gives exact control identity and read-back for free, but not for SWT custom tables; accepting OCR there, bounded by exact-match rules, is cheaper and more controllable than an end-to-end vision agent, at the cost of sensitivity to fonts, DPI and selection highlighting. Normalising the capture and testing against fixtures keeps that manageable.
- **Local OCR versus LLM extraction.** OCR plus an anchor-based parser is fast and private but limited to documents with recognisable section headings. An LLM would generalise across layouts but adds cost, latency, non-determinism and a data-leaving-the-machine question. Putting both behind the same validated model means the choice can change later without touching the flow.
- **Verify everything versus speed.** Reading back every field and re-selecting every created record from the Order roughly doubles the run time compared with a fire-and-forget script. For an accounting system I think that is the right side of the trade: the output is an evidence trail in which every value was read from the application, not assumed.
- **Literal PDF sequence versus the "natural" one.** Following the prescribed order (for example, VAT before New product, follow-up action for the Invoice) makes the run slower than a hand-tuned path but keeps the automation reviewable against the specification. Where Fakturama's behaviour forces a change in order, I would document it as a named deviation rather than silently reshuffle.

8. Risks and what I would check first

- Whether the Debtor editor's payment dropdown refreshes after a new payment method is created while it is open; if not, the method has to be created before the editor is opened.
- Whether the Order's Delivery address populates correctly after selecting a Debtor whose single address carries both roles.
- Date-widget segment behaviour under different locales.
- OCR accuracy on the selector dialogs at non-100% DPI scaling.
- Whether the Documents list can be searched reliably by Cust.Ref. for the restart check.

Testing would be layered: unit tests for the extractor and validator on fixture images, tests for the control-lookup and date logic against fakes, and live runs against a fresh, isolated Fakturama

workspace per run so each pass starts from an empty database and the “create” branches are actually exercised.